07 快速排序

Part II Sorting and Order Statistics

本章定位

Quicksort的分割只建立兩側與pivot的大小關係，還要遞迴才完成排序。本章從Lomuto不變量追到完整數值狀態，再把最壞時間、內部亂數的期望比較數與stack深度分開分析，特別處理重複鍵造成的退化。

前置知識

陣列分割、遞迴與期望值。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

本章路線

學習目標

- 實作 partition 並維持正確的分割不變量。
- 區分最壞、最好與期望快速排序。
- 分析隨機選 pivot 如何解除對抗性輸入。
- 7.1 Description of quicksort
- 7.2 Performance of quicksort
- 7.3 A randomized version of quicksort
- 7.4 Analysis of quicksort

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

7.1 Description of quicksort

Quicksort先選pivot，將其他鍵按與pivot的大小分開，再遞迴排序兩側。partition完成時只保證區域間的大小關係，兩側內部尚未排序。本節採1-index、Lomuto、最後位置r為pivot。

```
quicksort(A,p,r):
    if p>=r: return
    q=partition(A,p,r)
    quicksort(A,p,q-1)
    quicksort(A,q+1,r)
partition(A,p,r):
    pivot=A[r]; i=p-1
    for j=p..r-1:
        if A[j]<=pivot:
            i++; swap(A[i],A[j])
    swap(A[i+1],A[r])
    return i+1
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

空區間與單元素是基底；兩個遞迴子區間都排除q，所以嚴格縮小。若還把q放回同一側，遇pivot在端點就可能無限遞迴。

Lomuto的迴圈不變量

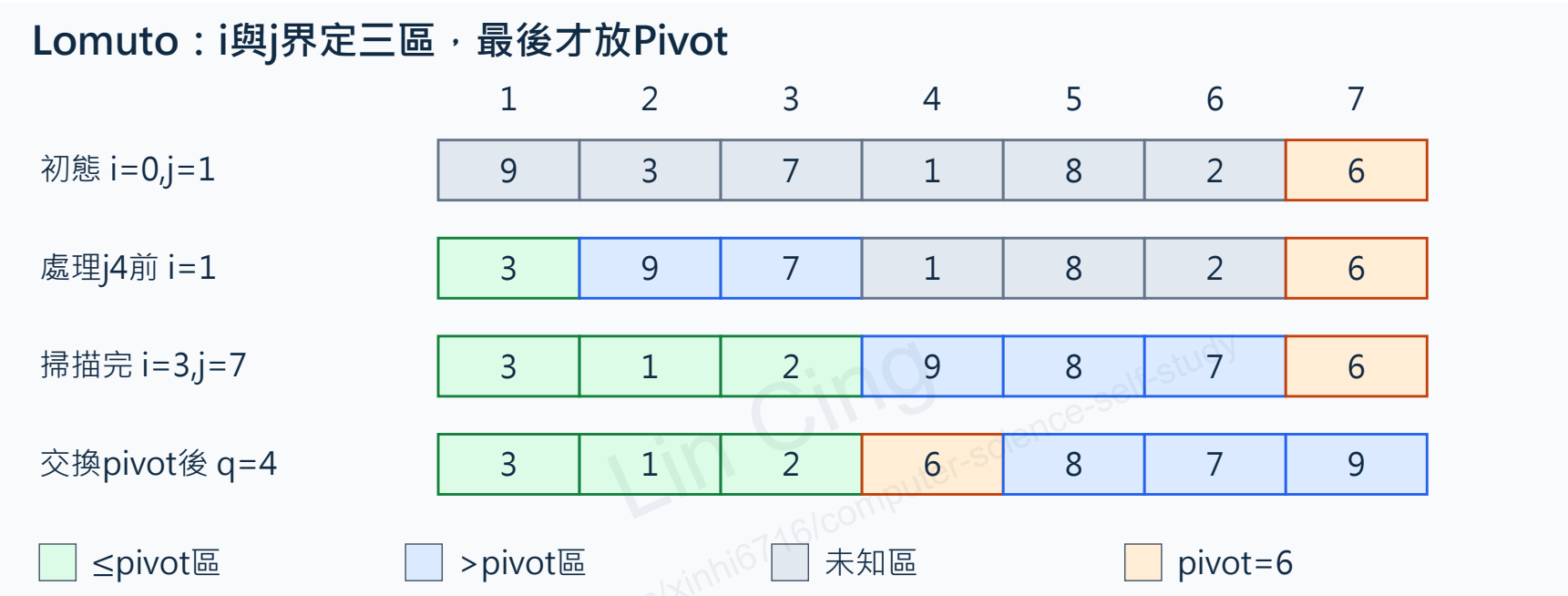
每次處理j前，A[p..i]≤pivot，A[i+1..j−1]>pivot，A[j..r−1]尚未處理，A[r]=pivot。空區間採通常的空集合慣例。

初始化i=p−1、j=p，前兩區都空。若A[j]≤pivot，i加1後與A[j]交換，新的小鍵移到≤區尾端；被換到j的舊值原屬>區，下一輪j加1後仍在正確區域。若A[j]>pivot，只增j，>區擴大。

結束j=r時沒有未知區。把pivot與A[i+1]交換，得到q=i+1，左區≤pivot、右區>pivot，pivot在q。交換保留元素多重集合，每一側長度小於原區間。以區間長度歸納，兩側排序後接上pivot，全段即排序正確。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Lomuto的迴圈不變量 （續）



Lin Cing
https://github.com/xinhi6716/computer-science-self-study

完整數值分割追蹤

A=[9,3,7,1,8,2,6]，p=1、r=7、pivot=6，初i=0。

j	判斷	i更新	交換後陣列
1	9>6	0	[9,3,7,1,8,2,6]
2	3≤6	1	[3,9,7,1,8,2,6]，swap1,2
3	7>6	1	[3,9,7,1,8,2,6]
4	1≤6	2	[3,1,7,9,8,2,6]，swap2,4
5	8>6	2	[3,1,7,9,8,2,6]
6	2≤6	3	[3,1,2,9,8,7,6]，swap3,6
結束	pivot放i+1	q=4	[3,1,2,6,8,7,9]，swap4,7

左[3,1,2]都≤6，右[8,7,9]都>6，內部並未排序。partition比較正好r−p次，時間Θ(r−p+1)、額外空間O(1)。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

7.2 Performance of quicksort

若每次pivot都是最小或最大，子問題長度0與n−1，partition做n−1次比較，總比較

$(n-1)+(n-2)+\dots+1=n(n-1)/2=\Theta(n^2)$。

例如已排序的n個不同鍵、固定取最後pivot，每次都是最大，這是一整族最壞輸入，不只是某個小例子的時間觀察。

上界也要涵蓋所有分割，而不是只算這個壞例子。令C(n)為最壞比較數，子問題長度a+b=n−1。歸納假設給

$C(a)+C(b)+(n-1) \le a(a-1)/2+b(b-1)/2+(n-1) = n(n-1)/2-ab \le n(n-1)/2$。

基例n=0、1皆為0，故所有輸入至多n(n−1)/2次比較，端點分割恰達界。每次partition固定開銷O(1)，每個被選pivot不再出現，總呼叫O(n)，故總時間$O(n+C)=O(n^2)$，配上上述族得最壞$\Theta(n^2)$。

若每次非空子問題的較大側最多ρn、固定ρ<1，每層所有partition處理量至多n，最長分支$O(\log_{(1/\rho)} n)$層，所以總O(n log n)。1:9分割仍是此情況，不需要恰好平分。

7.2 Performance of quicksort （續）

完全均分的n=2^k−1個鍵有k層，第d層剩餘處理量為n−(2^d−1)；前一半層數每層仍為Θ(n)，給Ω(n log n)，配上上界得此均分情況Θ(n log n)。這個下界來自該次遞迴工作量，沒有把比較排序的「至少某個輸入很慢」誤用成每個輸入都慢。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

重複鍵如何破壞二路分割

對全相等A=[5,5,5,5,5]，Lomuto每次判斷≤都成立，q=r，左側長n−1、右側0。即使每輪先隨機選pivot，換來的仍是相同5，總比較仍10；一般n為n(n−1)/2。

因此下一節的隨機O(n log n)分析先假設鍵互異。若要支援大量重複鍵的效率，可使用小於/等於/大於三路partition，整段等值區不再遞迴；第9章會完整列出區間邊界與rank換算。

常見的「隨機pivot解決一切最壞輸入」缺少鍵相異或適當重複鍵處理的前提。穩定性也是不同性質，quicksort的長距離swap一般會改變相等鍵identity順序。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

遞迴空間與只遞迴較小側

原偽碼的兩次遞迴最壞可有n層堆疊，即使partition原地進行，整個quicksort也不是無條件O(1)額外空間。

```
while p<r:
    q=partition(A,p,r)
    if q-p < r-q:
        quicksort-small-stack(A,p,q-1)
        p=q+1
    else:
        quicksort-small-stack(A,q+1,r)
        r=q-1
```

每次真正的遞迴只選較小側，長度≤(n−1)/2；較大側由while繼續，不保留新一層frame。每層遞迴規模至少減半，故最壞堆疊O(log(n+1))。所有partition仍會執行，時間最壞仍Θ(n^2)，不是藉減少stack就保證時間O(n log n)。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

7.3 A randomized version of quicksort

對目前A[p..r]，從p..r均勻隨機選k，先swap(A[k],A[r])再用相同Lomuto partition。後續每次在當前子區間重新均勻選擇。鍵互異時，pivot的秩對1..n均勻，任何固定輸入排列都有同一個期望分析。

這是對演算法內部亂數取期望，不要求使用者資料的排列均勻。某次執行仍可能一直抽到端點而花 $\Theta(n^2)$；下面證明期望值，尚未把它當成每次執行的保證或未證的尾機率界。

理想模型採條件於目前狀態仍均勻的抽樣。固定且公開的偽亂數序列是可重現的實作選擇，其對抗性保證必須另外說明，不能把形式上的random呼叫當作數學均勻性證明。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

7.4 Analysis of quicksort

令z1<z2<…<zn為排序後的不同鍵。對i<j，X_ij為zi與zj在任何一次partition中直接比較的指示變數。每對至多比較一次，因直接比較時至少一個是pivot，該pivot之後不再遞迴。

看秩區間Zi,j={zi,…,zj}。在其中任何鍵首次成為pivot前，兩端仍同屬某個子問題；區間外pivot只會整個保留這一段，不能把兩端分開。

若最先被選中的是zi或zj，該次partition直接比較兩端；若先選內部鍵，兩端被分至不同子問題，永不再比較。區間中每鍵等可能先成pivot，所以

`Pr(X_ij=1)=2/(j-i+1)`。

這個事件是「最先選中區間端點」，不是「區間任意元素先成pivot就會比較」。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

期望比較數的精確求和

總比較C=Σ_(i<j)X_ij。無論各X_ij是否獨立，期望線性性都給

E[C]=Σ_(i<j)2/(j−i+1)

=2Σ_(k=2)^n (n+1−k)/k

=2(n+1)H_n−4n，其中H_n=Σ_(k=1)^n1/k。

由ln(n+1)≤H_n≤1+ln n得E[C]=Θ(n log n)。加上partition迴圈與遞迴固定開銷同階，期望時間O(n log n)。不能以「所有比較事件獨立」計算變異數或高機率界，因它們並不獨立。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

四個秩的完整比較機率表

元素對	區間大小	比較機率
(1,2)、(2,3)、(3,4)	2	各1
(1,3)、(2,4)	3	各2/3
(1,4)	4	1/2

總期望=3+4/3+1/2=29/6，與2×5×H4−16=29/6一致。兩端1、4若先遇pivot2或3就分離，先遇1或4才比較；四種首選各1/4，所以機率1/2。

相鄰秩必比較不代表所有pair必比較。精確公式也不是每次執行的比較數，例如最壞4鍵需6次，29/6是對所有亂數選擇的平均。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

資料來源

書目

章節來源

Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 3rd ed., MIT Press, 2009，Chapter 07。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

參考資料與署名

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study